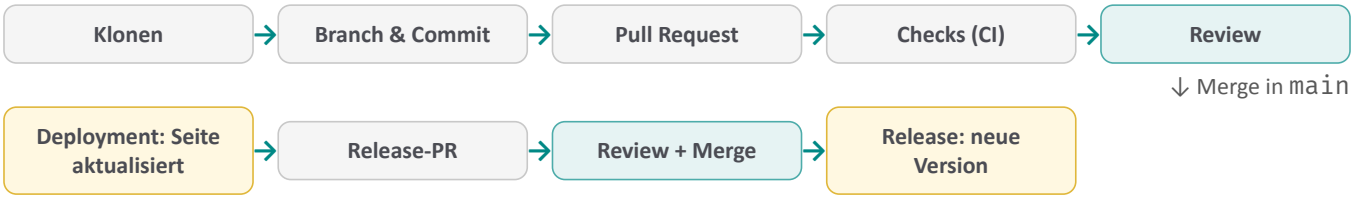
Git & GitHub — die Take-aways

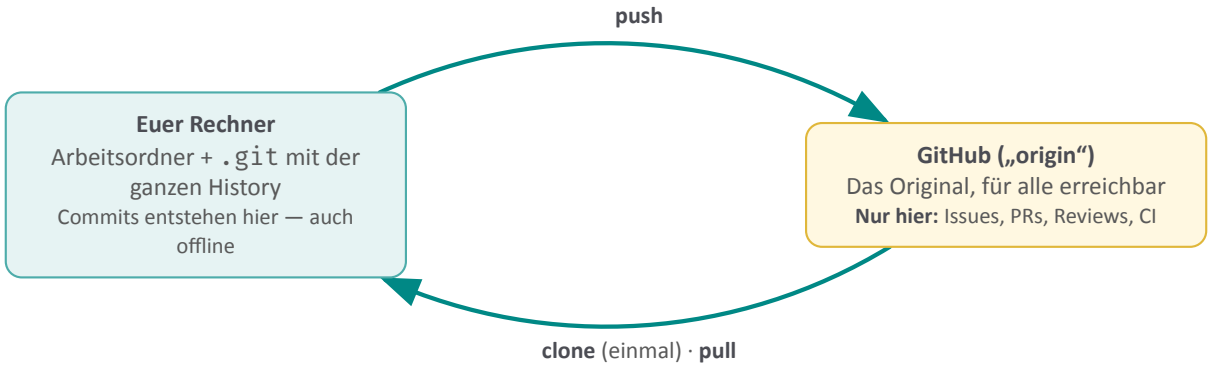
Repo `MeKo-Christian/workshop-teamwand` · Seite `meko-christian.github.io/workshop-teamwand` (nach jedem Merge, Version im Footer)

Der Weg einer Änderung



Niemand ändert `main` direkt — nicht ihr, nicht ein KI-Assistent, nicht die Admins. Jede Änderung nimmt diesen Weg, auch die kleinste. In echten Projekten gibt es meist zwei Umgebungen: **Staging** nach jedem Merge, **Prod** erst mit dem Release. Die Teamwand hat eine Seite für beides.

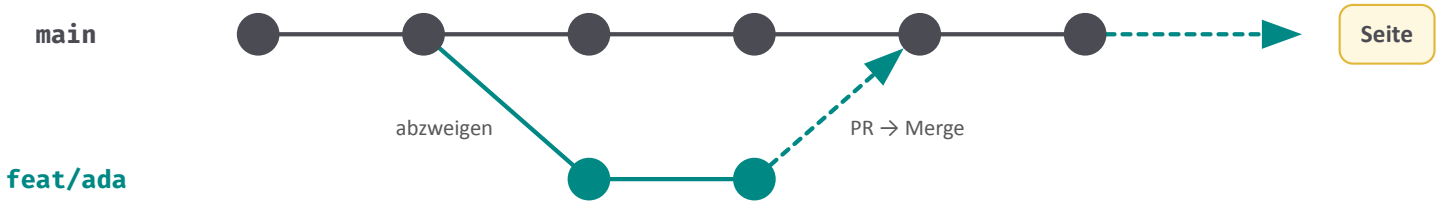
Zwei Kopien: euer Rechner und GitHub



- Nichts gleicht sich von selbst ab: **ihr ändert** → GitHub sieht es erst nach dem **push**. **Andere ändern** → ihr seht es erst nach dem **pull**.
- **Push abgelehnt?** GitHub hat auf dem Branch Commits, die ihr nicht habt → erst **pull**.
- **Ordner gelöscht?** GitHub bleibt unverändert, neu klonen. Verloren ist nur, was nie gepusht wurde.
- Ein Merge in `main` blockiert euren Push nicht; `main` holt ihr eigens in euren Branch.

Faustregel: vor der Arbeit pull, nach dem Commit push.

`main` und Branch



- `main` = der **abgestimmte Stand**, den ihr beim Klonen bekommt. Der Name ist frei wählbar (früher meist `master`).
- **In diesem Repo** geht jeder Merge in `main` automatisch auf die Seite (in echten Projekten meist auf Staging).
- Neue Arbeit startet immer vom neuesten `main` → erst pull.
- **Ein Issue = ein Branch = ein PR:** klein, gut zu reviewen.
- Name: Typ + Kurzbeschreibung, z. B. `feat/ada`.
- Ein Branch kostet nichts (nur ein Name), entsteht lokal, wird nach dem Merge gelöscht. Schiefgegangen? Wegwerfen.

Der PR-Titel wird zum Changelog

Beim Merge wird der PR-Titel **wörtlich** der Commit auf `main` und landet in den Release-Notes. Darum hat er eine feste Form, und die CI prüft sie:

`typ: was sich ändert`

Gut

```
feat: add anna to the
team wall
fix: keep long mottos
inside the card
```

Fällt durch die CI

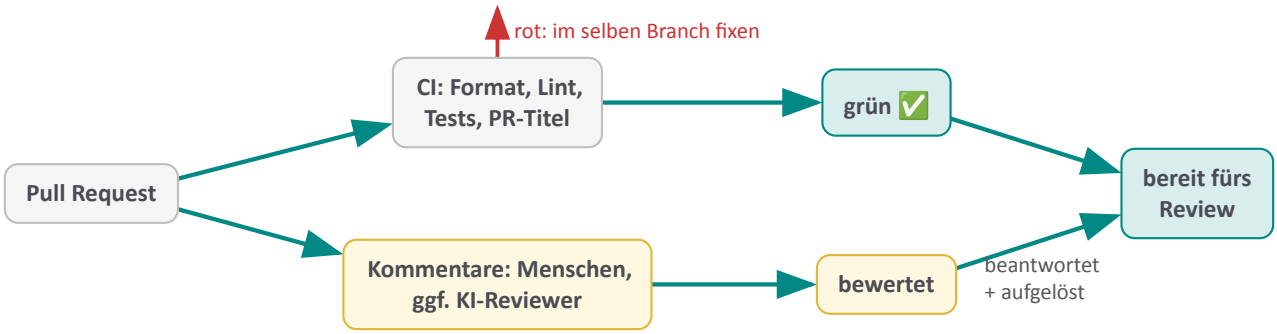
```
Update team.js
fix stuff
```

Formal ok, sagt nichts

```
feat: changes
```

`feat`	neue Funktion	`docs`	nur Dokumentation
`fix`	Fehler behoben	`refactor`	Umbau ohne neues Verhalten
`test`	nur Tests	`chore`	Sonstiges (Abhängigkeiten, Aufräumen)

Checks



- **CI** (Continuous Integration) prüft **automatisch** bei jedem push: Formatierung, Lint, Tests, PR-Titel. **Rot = kein Merge.**
- Rot wird **im selben Branch** repariert, nie werden Regeln abgeschaltet. Rot ist gut: der Fehler ist gefunden, bevor ihn jemand sieht.
- Kommentare **bewerten**, nicht blind umsetzen — auch die eines KI-Reviewers (z. B. GitHub Copilot, falls eingeschaltet).
- Jeder Thread wird beantwortet und aufgelöst, sonst lässt sich nicht mergen. Die Freigabe gibt immer ein Mensch.

Commit und Pull Request

Ändern (Editor) → **Vormerken: `git add`** → **Committen: `git commit`** → **Hochladen: `git push`** → **Pull Request auf GitHub**

- **Commit** = gespeicherter Stand mit Nachricht, Autor, Zeit und ID. Entsteht nur auf eurem Rechner.
- `git add` wählt aus, **was** in den Commit kommt. `git status` zeigt jederzeit Branch, Änderungen und Vorgemerktes.
- **Pull Request** = die Bitte „nehmt meinen Branch in `main` auf“ — nur auf GitHub: Änderungen (**Files changed**), Checks, Kommentare, Freigabe, Merge.
- Neue Commits auf dem Branch landen nach dem push automatisch im selben PR.

Review: Vier Augen

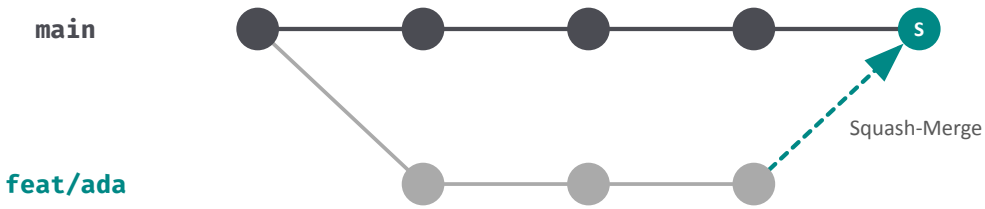
Ihr als Reviewer:in

- Passt der PR zum Issue — und **nur** dazu?
- Selbst ansehen, gern lokal ausprobieren.
- Ist der Titel ein guter Changelog-Eintrag?
- Konkret kommentieren: **was** und **warum**. Änderungen anfordern ist nicht unhöflich.

Ein KI-Reviewer (falls eingeschaltet)

- Findet Muster, schnell und gründlich.
- Kennt euer Ziel nicht und kann irren.
- **Gibt nichts frei:** Das macht immer ein Mensch, nie der Autor selbst. Approve heißt: ich stehe mit dafür ein.

Merge und Konflikt



= ein Commit mit PR-Titel

- **Squash-Merge:** der ganze PR wird **ein** Commit S auf `main`, mit dem PR-Titel als Nachricht. `main` bleibt eine gerade Linie, egal wie es im Branch aussah.
- Jeder Merge ändert `main`: mergt **nacheinander**, die anderen ziehen nach. Danach lokal **pull**, sonst kennt euer Rechner den neuen Stand nicht.

Konflikt: zwei PRs ändern **dieselbe Stelle**. Wer zuerst merged, gewinnt, der andere zieht nach.

```
<<<<<<< main
  { login: "anna", … },
=======
  { login: "ben", … },
>>>>>>> mein-branch
```

Was richtig ist, wisst ihr

Git kann nicht entscheiden, welche Zeile gilt. In der Teamwand: **beide** behalten, alphabetisch sortiert.

Selbst: Resolve conflicts im PR, Markierungen löschen → **Mark as resolved** → **Commit merge**. **KI: „Löse den Konflikt, behalte beide Einträge und erklär mir, was du gemacht hast."**

Release: Versionen aus PR-Titeln



release-please hält nach jedem Merge **einen** Release-PR offen:

- Changelog aus euren **PR-Titeln**
- neue Versionsnummer
- Merge des Release-PRs → **neue Version** (Footer der Seite, GitHub-Release)

In echten Projekten geht genau dieser Stand dann nach Prod.

chore(main): release 0.2.0

```
## 0.2.0

### Features
* add anna to the team wall
* add ben to the team wall
* add carla to the team wall
```

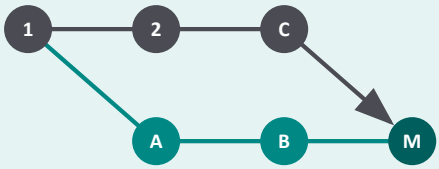
Und wenn die neue Version kaputt ist?

Kein Hotfix per Hand. Die Änderung per **Revert-PR** zurücknehmen und neu releasen — gleicher Weg, auch unter Zeitdruck.

Merge oder Rebase? Euren Branch auf `main` nachziehen

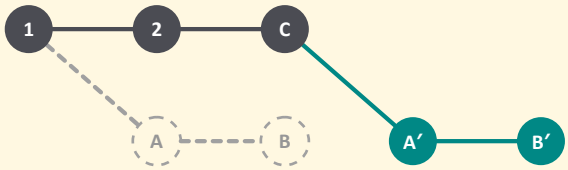
Euer Branch ist hinter `main`, weil inzwischen C gemerged wurde. Zwei Wege:

Merge — der sichere Standard



Ein Merge-Commit M verbindet beide Linien; A und B bleiben unverändert. Das macht auch „Update branch" auf GitHub.

Rebase — neu aufsetzen



Eure Commits werden auf das neue `main` neu abgespielt → **neue** Commits A′, B′. Danach braucht der push `--force-with-lease`.

Goldene Regel

Nie Commits rebasen, die andere schon haben — sonst gibt es doppelte Commits. `main` ist gegen force push gesperrt. Der Code ist nach beiden Wegen derselbe, nur die History nicht.

Etappe für Etappe: selbst oder mit KI-Assistent

Einmalig vorab: `git config --global user.name "Anna Muster"` und `git config --global user.email "eure@mail.de"`. Beim ersten push fragt Git im Browser nach dem GitHub-Login. Beispiele für Issue 7, Login `anna`.

Etappe	Selbst: Befehl oder Klick	Oder: sagt eurem KI-Assistenten	Selbst prüfen
1 · Klonen	`git clone https://github.com/MeKo-Christian/workshop-teamwand` `cd workshop-teamwand` `git log --oneline -5` · Issues auf GitHub ansehen	**„Klone MeKo-Christian/workshop-teamwand in meinen Projektordner.“ „Welche Issues sind mir zugewiesen?“**	`index.html` im Browser öffnen
2 · Branch & Commit	`git switch -c feat/7-teamwand-anna` · Zeile in `team.js` einfügen · `git add team.js` `git commit -m "feat: add anna to the team wall"`	**„Leg einen Branch nach AGENTS.md an, erstelle meine Karte (Name, Emoji, Motto) und committe. Noch nicht hochladen.“**	Karte lokal sichtbar? Branch auf GitHub? (noch nicht)
3 · Push & PR	`git push -u origin feat/7-teamwand-anna` · **Compare & pull request** · Titel `feat: add anna to the team wall`, Text `Closes #7`	**„Lade meinen Branch hoch und öffne einen PR mit Conventional-Commit-Titel. Gib mir den Link.“**	Titel verständlich? Nur eure Zeile geändert?
4 · Checks	Checks im PR abwarten · rot: **Details**, Log lesen, korrigieren, `git commit -am "fix: …"` `git push`	**„Sind die Checks grün? Falls rot: Lies das Log und behebe den Fehler, ohne Regeln abzuschalten.“**	Kommentare beantworten und auflösen
5 · Review	**Reviewers** im eigenen PR setzen · fremden PR: **Files changed**, lokal `git fetch` `git switch <branch>` · **Review changes**	**„Fordere ein Review von der Person an, die in meinem Issue steht.“ „Hol mir diesen PR und fass zusammen: Passt er zum Issue und zu AGENTS.md?“**	Fremden PR **selbst** freigeben
6 · Merge & Deployment	**Squash and merge** · `git switch main` `git pull` `git branch -D feat/7-teamwand-anna`	**„Ist mein PR freigegeben und grün? Dann merge ihn.“ „Hol den neuesten Stand von main und lösch meinen alten Branch lokal.“**	Die Seite zeigt eure Karte
7 · Release	Release-PR öffnen, Changelog lesen, gemeinsam freigeben und mergen · **Actions** und **Releases** ansehen	**„Zeig mir den offenen Release-PR und fass den Changelog zusammen.“**	Neue Version im Footer der Seite

Wenn es hakt: `git status` zeigt, wo ihr steht · Branch nachziehen: **Update branch** im PR · **„Erklär mir, was du gerade gemacht hast und warum.“**

Eure Regeln auf einen Blick

- Niemand ändert `main` direkt.
- Ein Issue, ein Branch, ein kleiner PR.
- Vor der Arbeit pull, nach dem Commit push.
- PR-Titel: `typ: was sich ändert`.
- Rote CI wird repariert, nicht abgeschaltet.
- Jeder Kommentar wird bewertet und beantwortet.
- Das Review ist ein Mensch, nie der Autor.
- Neue Version nur über den Release-PR; Rollback per Revert-PR.

Ob ihr tippt oder ein KI-Assistent: Verantwortlich seid ihr — und das Review ist eure Unterschrift.

Weiterüben: Issues mit Label `bonus` (kleine Features mit Test, gleicher Weg) · Für euer eigenes Projekt: `main` schützen, CI als Pflicht, Squash-Merge